
DEEP LEARNING PROJECT - GROUP 8

KEYPOINT DETECTION

TECHNICAL REPORT

Aarhus University

Aarhus University

Aarhus University

December 5, 2025

ABSTRACT

Keypoint detection is a task that aims to find keypoints in an image that correspond to human joints or body parts. In image-to-image keypoint detection, models output heatmap images that predict where the image likely contains a specific keypoint. We implemented and iteratively improved a model that aims to predict 17 keypoints. We managed to improve the model from only detecting the background to being able to detect specific keypoints in an image. This was mainly achieved by improving the information present in the data, the information retained by the model, and the loss metric used.

1 Introduction

Keypoint detection as a field is concerned with detecting keypoints from visual data, for example an image or a video [Vina \[2025\]](#). The data is typically images or video frames, and the keypoints can be either 2- or 3D. The object whose keypoints are to be detected is usually a human but does not have to be. Keypoint detection, and by extension pose estimation, has various applications, such as in athletics, physical therapy and surveillance [Vina \[2025\]](#). In this project our motivation to work with keypoint detection is educational. We want to learn about image segmentation for keypoint detection and investigate how different design choices impact performance.

The objective of our work is to implement a model that can predict up to 17 human keypoints in an image. The keypoints are body parts such as the left shoulder, the right knee, and the nose. The model shall for each image output 18 images. 17 of the 18 images shall represent keypoints and the 18th image shall represent the background.

2 Related work

For keypoint detection, there are two main approaches: regression methods and image-to-image (heatmap) methods [Zheng et al. \[2023\]](#). Regression methods aim to output keypoint coordinates based on an input image. Meanwhile, image-to-image methods aim to produce an image estimate showing 2D Gaussian peaks at keypoint locations in an image. We chose to implement an image-to-image approach, as we expected the results to be more informative for us to base further experiments on.

An example of such an image-to-image keypoint detection model comes from [Wei et al. \[2016\]](#), which addresses the vanishing gradient problem in fully convolutional models. This paper explains an approach which is based on performing multiple convolutions resulting in an image for each keypoint and one image for the background. These images show the estimated probability that each pixel belongs to a certain keypoint. This turns the problem into one of image segmentation, where the task is to categorize each pixel as belonging to a certain keypoint or not.

A popular model architecture for image segmentation is the U-Net [Ronneberger et al. \[2015\]](#). The key change in the U-Net architecture is that it adds skip connections between the encoder and decoder layers in the encoder-decoder structure. This helps preserve spatial information within the image and avoids vanishing gradients.

3 Methods

In this section, we will start with describing the dataset and preprocessing that we used, followed by a description of the initial baseline model we made, and then describing the experiments we performed to improve the model.

For each model, we plot the training and validation loss alongside a sample of the gradients and activations of the model to evaluate the model’s training process. This helped us determine the next steps for our experiments and detect potential under- and overfitting.

To evaluate the final out-of-sample performance of a model once training is done, we calculate the Object Keypoint Similarity (OKS) [Dutta and Dawn 2023]. These results and the training evaluation are shown in the results section.

3.1 Dataset

We used a subset of the Keypoints 2017 split of the COCO (Common Objects In Context) image dataset [Lin et al. 2015] consisting of randomly chosen images with annotations that contain at least one keypoint. We used 500 of these images for our experiments, and 2000 for the final model, in order to run our experiments in a timely fashion. We then cropped out keypoint-annotated people in each image based on the provided bounding boxes. Any grayscale images present in the dataset were converted to RGB by copying the image along 3 channels. An 80%/10%/10% split was used in each experiment for the train, validation, and test sets respectively.

The keypoint annotations consist of a 51 length vector such that the i -th keypoint’s coordinates x, y and visibility flag i is described by $(x, y, f)_i = (i, i + 1, i + 2)$. For cropping we use bounding box annotations, which consist of (x, y, w, h) where (x, y) is the coordinate of the top left corner of the bounding box and (w, h) is the width and height of the bounding box.

To create the target labels for an image we create a two-dimensional zero-initialized array with the same width and height as the image, for each keypoint in the image, where the value at the coordinate of the keypoint is set to 1. This gives us a three dimensional tensor of shape (height, width, 17). We then add a background channel to the tensor which is 1-initialized, but the value at the coordinate of each keypoint is set to 0.

3.2 Visualization

To visualize keypoint predictions and labels we dampen the red channel in an image by 80% and then overlay keypoint predictions on top of the dampened red channel, this causes visualizations of keypoints to be blue-tinted.

3.3 Baseline

For our project, the task is similar to an image segmentation task. Hence, we wanted to work towards eventually implementing a U-Net architecture [Ronneberger et al. 2015]. For the initial baseline, we decided to start with a simple image-to-image encoder-decoder architecture. Our chosen encoder was the VGG16 model pretrained on the imagenet dataset. The decoder architecture consisted of 5 blocks of a transposed convolution layer followed by two convolution layers decreasing the number of 3x3 filters from 512 to 64 by a factor of 2 each block. Padding was used to maintain image size. This decoder structure roughly mirrors the VGG16 encoder and ensures equal sized outputs so the output keypoint images can be mapped onto the input image. The final layer uses a softmax activation function along the keypoint channel to output keypoint probabilities for each pixel.

The model was trained for 20 epochs with a batch size of 8. Training was slow, so initial tests were performed on smaller subsets of the data. The initial chosen loss function was Mean Squared Error (MSE). MSE loss was used because we consider this to be the simplest choice. This will be updated in further experiments as we took more of an image segmentation approach. The model was trained using Stochastic Gradient Descent (SGD) with a learning rate of 10.0 which is very high since we noticed a certain pattern during training that will be described later.

The initial model does learn a solution, but due to the small amount of information present in the target images that mapped keypoints onto an individual pixel, the model simply learned to predict background for each pixel. But it gave us a working model to start improving upon.

3.4 Experiments

With the baseline model learning to predict only background, we had a functioning model to start improving upon. In this section, we will describe the various experiments we did to improve upon the model. For each experiment, hyperparameter tuning was performed through a random search on parameters based on common values used in other

implementations. Moreover, for each experiment, we used a subset of 500 images to be able to run our experiments within a shorter time and get useful results.

3.4.1 Gaussian Peaks

Initially, only a single pixel in the target array was set to 1 for each keypoint (the pixel at the keypoint position) and the arrays are quite big, roughly $\approx 100 \times 100$, so the model quickly finds an optimal strategy of predicting 0 for every pixel in the keypoint channels where the target labels consist of almost entirely 0's, and 1 for every pixel in the background channel where the target label consists of almost entirely 1's.

To alleviate this problem, we instead put a gaussian distribution of values between 0-1 centered around the keypoints coordinates, such that the pixel at the coordinate of the keypoint has value 1 and then pixels gradually go to 0 as you go away from the pixel at the keypoint coordinate. This approach was based on [Wei et al. \(2016\)](#) and [Zheng et al. \(2023\)](#).

We initially had the gaussian distributions be fixed size, but this lead to bad target labels as the gaussian distribution would fill the entire image for small images and only a tiny bit for large images, we thus switched to scaling the size of the gaussian distribution depending on the size of the image.

We also switched to using the Adam optimizer, since this is a more advanced and effective optimizer than SGD.

3.4.2 U-Net Connections

With the addition of Gaussian peaks to the target images, our model still learned to predict background for all pixels [\(4.2.1\)](#). But because this approach gives the model more information to learn from and is commonplace in image to image keypoint detection, we continued to use it in future experiments.

For our next experiment, we decided to add skip connections similar to the U-Net architecture [\(Ronneberger et al. 2015\)](#). This meant adding connections between the convolutional layers before each pooling operation in the encoder and connecting them to the convolutional layers after each transposed convolutional layer in the decoder.

The reasoning behind adding these connections is that they should help the model preserve spatial information present in the image that is lost during pooling while still learning important features through the encoder. Moreover, U-Net is an architecture that originally achieved great performance on image segmentation tasks.

The new U-Net model was trained using the Adam optimizer with an initial learning rate of 0.0002 for 20 epochs with a batch size of 8.

3.4.3 Batch Normalization

After adding U-Net connections, the model did not improve [\(4.2.2\)](#), but we kept using the U-Net architecture in the future experiments as it is known to do well in image segmentation tasks.

We observed that the U-net model had problems with exploding activations [\(6\)](#) and vanishing gradients, resulting in the model not learning. Because of this, our next experiment was to add batch normalization.

Batch normalization was performed before the activation function of each convolutional layer in the decoder, except the last layer. Hyperparameters were left unchanged.

3.4.4 Loss Function

After adding batch normalization, we observed that the model improved significantly [\(4.2.3\)](#), so we added batch normalization to all future experiments.

The next experiment involved improving our loss function. In the experiments so far, we used the MSE loss as it was the simplest loss to start with. However, other convolutional pose estimation models use different loss functions such as the cross entropy loss [\(Ronneberger et al. 2015\)](#) or the Dice loss [\(Azad et al. 2023\)](#). We decided to implement two versions of our U-Net model with batch normalization, one using the weighted cross entropy loss and the other using the Dice loss.

The weighted cross entropy loss used weights that were inversely relative to the average number of pixels of each keypoint. Initially, we implemented them as the direct inverse, but we tuned them to get better results. The idea is that these weights help the model focus on classes in the segmentation task that are less present than other classes.

The Dice loss is based on the Dice coefficient, which is a measure of similarity between two datasets. The Dice coefficient computes the overlap of the predicted and target segmentation mask divided by their sum for each keypoint

and then averages them over all keypoints. The Dice loss is calculated as follows:

$$L_{dice} = 1 - \frac{1}{C} \sum_{c=0}^{C-1} \frac{2 \sum_{n=1}^N t_n^c y_n^c}{\sum_{n=1}^N (t_n^c + y_n^c)}$$

where y_n^c is the n th predicted segmentation mask for class c and t_n^c is the n th target segmentation mask for class c . It is averaged over classes.

Both of these losses are a popular choice in case of imbalanced data. In our case, the dataset is imbalanced due to the background class typically containing many more pixels than the individual keypoint classes. Hence, a loss that is intended to be good for imbalanced data should be helpful.

The model with weighted cross entropy was trained using the Adam optimizer with an initial learning rate of 0.0002 for 20 epochs with a batch size of 8. The model with Dice loss was trained using the Adam optimizer with an initial learning rate of 0.0005 for 25 epochs with a batch size of 8.

3.4.5 L2 Regularization

After trying both weighted cross-entropy loss and Dice loss we decide to continue with Dice loss as it had slightly better prospects (4.2.4).

Our next experiment was adding L2 Regularization to the model. L2 Regularization is typically added to avoid overfitting. We did not observe overfitting, but our model kept going into suboptimal minima, so we tried L2 regularization to see if it would alleviate this problem. We added L2 regularization by adding weight decay to the Adam optimizer. The regularization parameter was tuned and set to 0.0001.

3.4.6 Increasing Dataset

After adding L2 regularization, we observed that the model improved slightly (4.2.5), so we added L2 regularization to the final experiment as well.

For our final experiment we increased the dataset size from 500 to 2000 and the batch size from 8 to 32, hoping that our model would generalize further and improve. It was run using the Adam optimizer with an initial learning rate of 0.0002 and regularization parameter of 0.0001 for 30 epochs.

4 Results

This section will cover the relevant results of each experiment that inspired the next steps in our project. We will not report every metric saved during training to save space in the report and focus on the metrics that were most relevant. Other metrics can be found in the relevant notebook files.

4.1 Baseline Results

As shown in Figure 1 the loss quickly decreases to near zero. This is due to the model simply learning to predict background for all pixels.

Figure 2 shows the output of an out-of-sample input image. It can be seen that the the model predicts nothing for the keypoints, and that the aggregated target image (all label) contains very little information as it is hard to even spot the keypoints, which consist of single pixels.

The test loss was 0.0, and the OKS was 0.007.

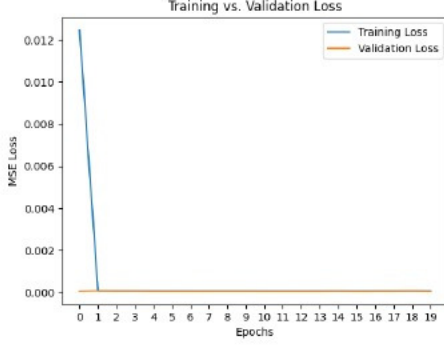


Figure 1: Training loss (blue) versus validation loss (yellow) for the baseline model.



Figure 2: Overview of the output keypoint images of the baseline model for a single input image.

4.2 Experiment Results

4.2.1 Gaussian Peaks

The Gaussian peak experiment had the same results as the baseline model w.r.t. the loss and performance.

4.2.2 U-Net Connections

The model still exhibits the same behaviour as before, with the loss quickly approaching zero as the model learns to predict all background. However, Figure 3 shows that the activations of the model are exploding further into the model. This is what caused us to apply batch normalization next.

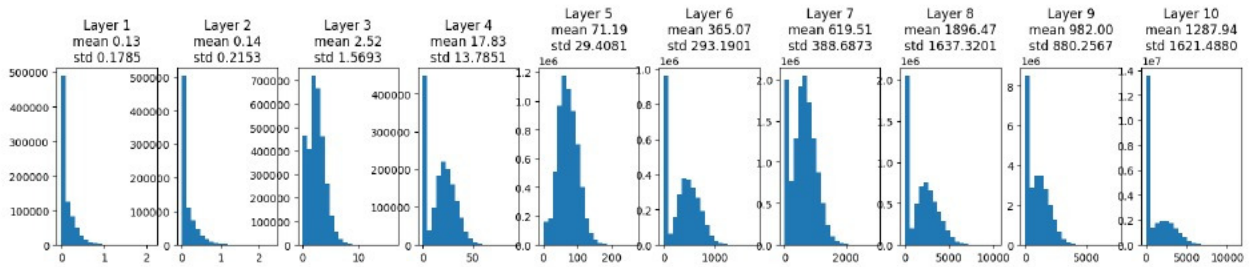


Figure 3: Activations for some initial layers of the U-Net model.

4.2.3 Batch Normalization

Using batch normalization, the model started making predictions deviating from outputting zero for all pixels in the heatmaps of the 17 body parts and ones for the pixels in the background. It learned to make seemingly correct guesses on some body parts while outputting only rough areas for other body parts, as can be seen in Figure 5. The new model has much smaller activation values than the corresponding model without batch normalization, as can be seen in Figure 3 and Figure 6.

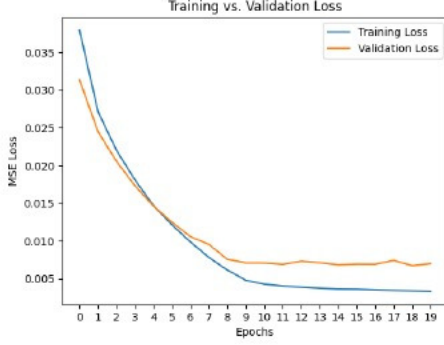


Figure 4: Training loss (blue) versus validation loss (yellow) for the U-Net model with batch normalization.

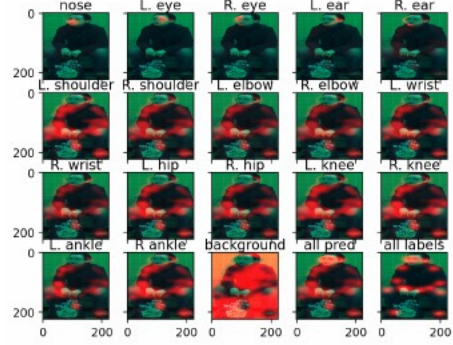


Figure 5: Overview of the output keypoint images of the U-Net model with batch normalization.

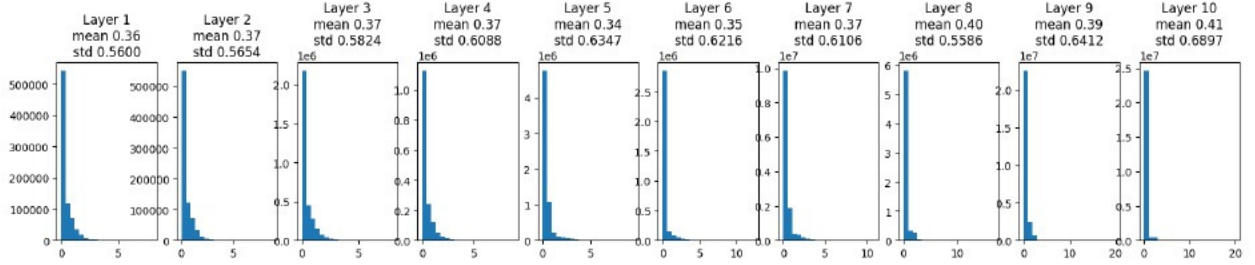


Figure 6: Activations for some initial layers of the U-Net model with batch normalization.

4.2.4 Loss Function

Weighted cross entropy The model using the weighted cross entropy loss gave a more promising loss curve as shown in Figure 7. Moreover, Figure 8 showed that the model was learning to predict more than just background. However, the model only slightly learned to distinguish the different keypoints. Each image shows that the model predicted regions near correct keypoints, but it would predict the same regions for all keypoints.

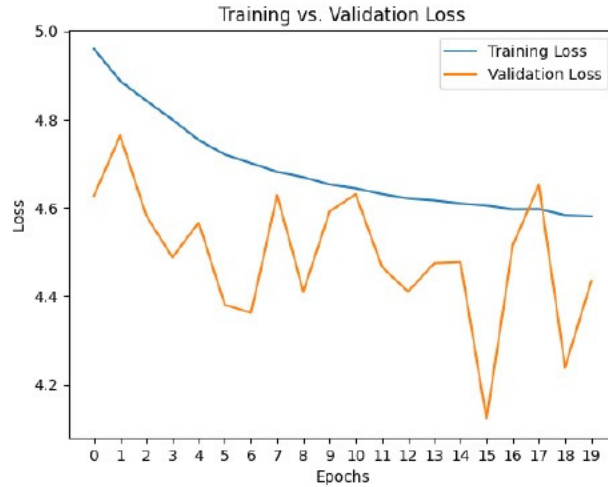


Figure 7: Training loss (blue) versus validation loss (yellow) for the U-Net model with weighted cross entropy loss.

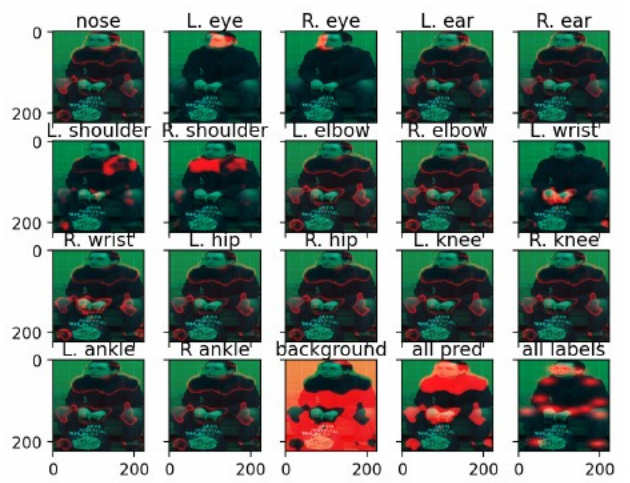


Figure 8: Overview of the output keypoint images of the U-Net model with weighted cross entropy loss for a single input image.

Dice loss The model using the Dice loss also gave a promising loss curve as shown in Figure 9. Moreover, Figure 10 showed that this model was learning to predict specific keypoints and output them in the right channel. However, this only worked for certain keypoints, such as the eyes, ears and shoulders, but not others, such as the elbows and wrists.

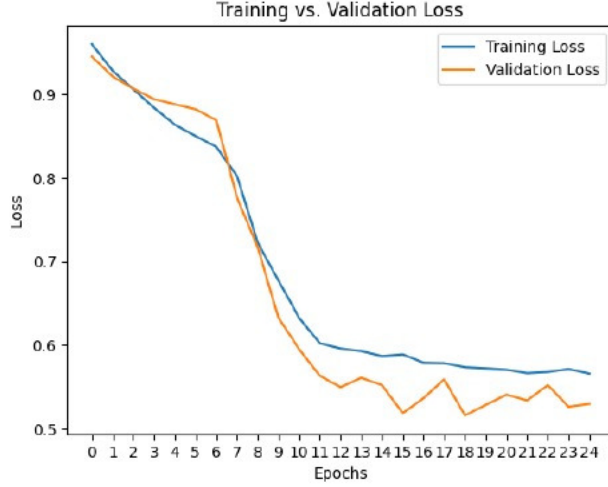


Figure 9: Training loss (blue) versus validation loss (yellow) for the U-Net model with Dice loss.

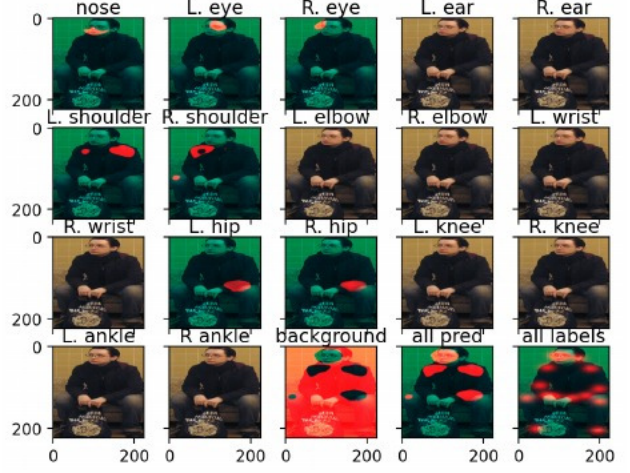


Figure 10: Overview of the output keypoint images of the U-Net model with Dice loss for a single input image.

4.2.5 L2 Regularization

Regularization seemed to perform slightly better than the corresponding model without regularization. The difference seemed to be that the model with regularization was slightly better at identifying specific body parts, like the ears. As can be seen in Figure 11 the model is seemingly not fully trained since the loss function appears to be decreasing. In this case we used visual inspection to analyse the output images, and even though the loss function could be decreased further, we found that training the model further would make its output worse by causing it to only predict certain keypoints and ignore all other channels.

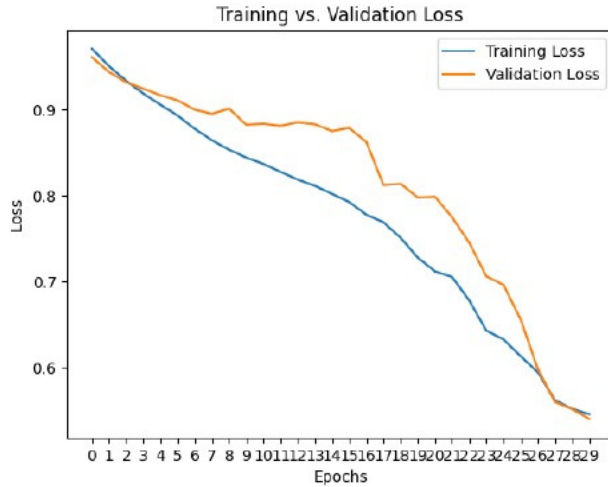


Figure 11: Training loss (blue) versus validation loss (yellow) for the model with regularization and Dice loss.

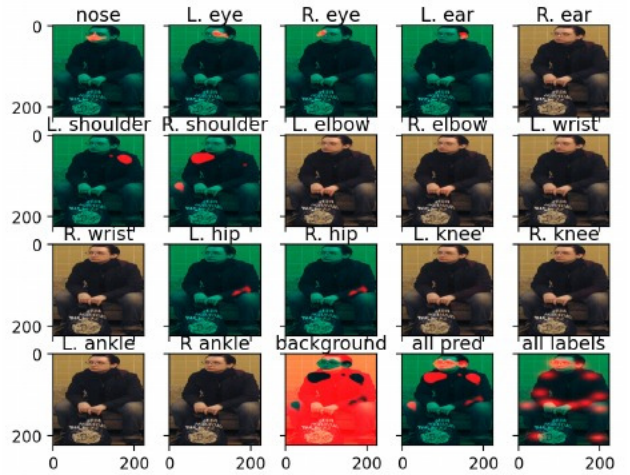


Figure 12: Overview of the output keypoint images of the model with regularization and Dice loss.

4.3 Final Model Performance

The final model was trained for 30 epochs on a training set of 1600 images. The model learns to predict head-keypoints well as seen on figure 13 but makes very general predictions on body-keypoints as seen on figure 14. The mean OKS achieved on the test set by the final model was 0.00453.

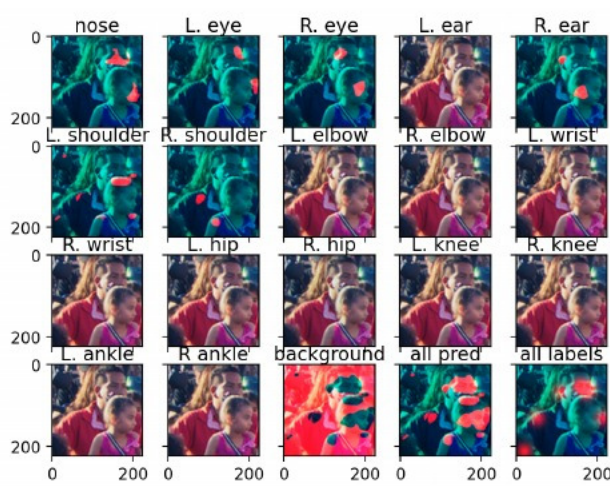


Figure 13: Overview of the output keypoint images of the final model for a single input image.

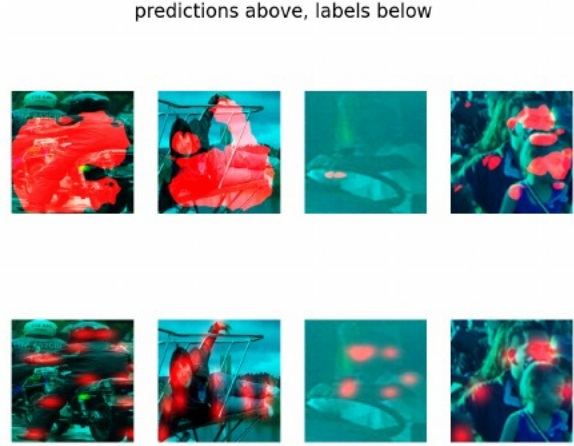


Figure 14: Overview of the aggregated keypoint predictions and labels for four images, for the final model.

5 Discussion

5.1 Conclusion

To conclude, we managed to improve our model from predicting only background to managing to predict specific keypoints within an image. Its performance was still limited to only a few specific keypoints, mainly around the face, while other keypoint information seems to get lost.

The main takeaways were that the model needed to retain enough information to do proper keypoint estimation and that the loss metric needs to be well adjusted to the task. This meant that the target images needed more information than individual pixels and the model needed the U-Net connections to retain spatial information in the images. Moreover, choosing a loss function that is well suited to both the dataset and the task at hand made a big difference. Also, ensuring an effective learning process through things like batch normalization has a big impact on model performance, which was expected but interesting to see in practise.

5.2 Limitations/Challenges

There were also some challenges and limitations with our project. The main initial challenge was that ensuring the data preprocessing worked properly and gave us usable images for our model ended up taking a lot more time than expected, as we found parsing the annotation format difficult and needed time to put the keypoint coordinates into images.

A limitation for our project was that we chose to use the Google Colab GPU resources, since we did not have sufficient computational resources available ourselves. This meant we were limited in the resources we could use within a certain period and for how long we could leave a model to run unattended (90 minutes according to Google). This also meant that we had to keep the size of our model and our dataset limited in a way appropriate to the resources available us. We suspect that limited model complexity may have limited the final results we were able to achieve, but our project still shows an iterative improvement of our baseline model.

A last limitation is that we were unsure of an appropriate performance metric to use. We chose the OKS, but this required a translation from output images to keypoint coordinates. We did this by taking the maximum value, but we suspect that this was too inaccurate for the metric to be informative. We also suspect there was an error in the way we calculate OKS. However, the output images still show our model's results and visual inspection was thus used as our main performance metric.

References

- Abirami Vina. What is pose estimation and where can it be used?, May 2025. URL <https://www.ultralytics.com/blog/what-is-pose-estimation-and-where-can-it-be-used> Accessed: 2025-11-27.
- Ce Zheng, Wenhan Wu, Chen Chen, Taojiannan Yang, Sijie Zhu, Ju Shen, Nasser Kehtarnavaz, and Mubarak Shah. Deep learning-based human pose estimation: A survey. *ACM computing surveys*, 56(1):1–37, 2023.
- Shih-En Wei, Varun Ramakrishna, Takeo Kanade, and Yaser Sheikh. Convolutional pose machines, 2016. URL <https://arxiv.org/abs/1602.00134>
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.
- Tapas Dutta and Kunal Dawn. Object keypoint similarity in keypoint detection, Aug 2023. URL <https://learnopencv.com/object-keypoint-similarity/#Per-Keypoint-Standard-Deviations-Initialized-from-COCO>
- Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft coco: Common objects in context, 2015.
- Reza Azad, Moein Heidary, Kadir Yilmaz, Michael Hüttemann, Sanaz Karimijafarbigloo, Yuli Wu, Anke Schmeink, and Dorit Merhof. Loss functions in the era of semantic segmentation: A survey and outlook. *arXiv preprint arXiv:2312.05391*, 2023.